Email: John.dixon@hull.ac.uk
Meetings: https://tinyurl.com/jdixonhull
Feedback: https://tinyurl.com/jdhullfeedback
Office: RBB-315

Resource Oriented Architectures

Distributed Systems Programming

Dr John Dixon

UNIVERSITY of HULL

Overview

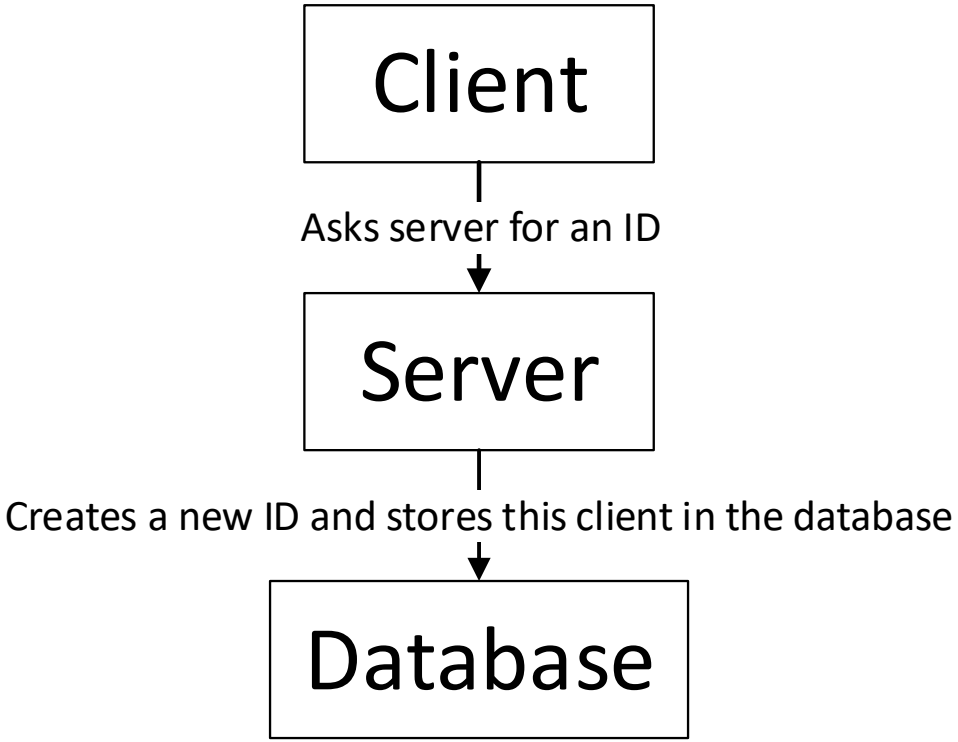
We are looking at these topics today:

- Stateful vs. stateless services/servers
- Resource Oriented Architectures
- RESTful services
- Web API (briefly)
- Using REST via HTTP principles

UNIVERSITY of HULL

Stateless Servers

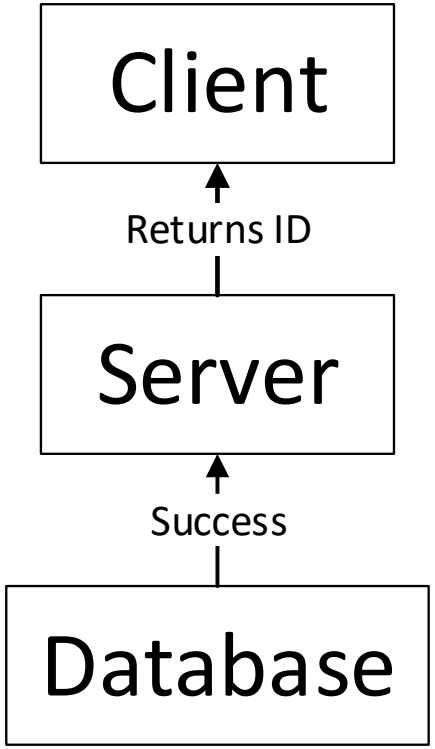
- An example:



UNIVERSITY of HULL

Stateless Servers

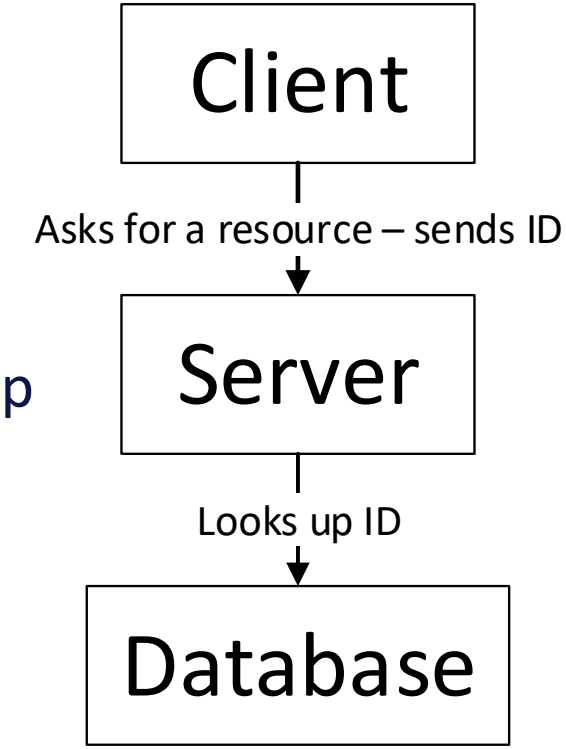
- An example:
- Server now forgets about the client



UNIVERSITY of HULL

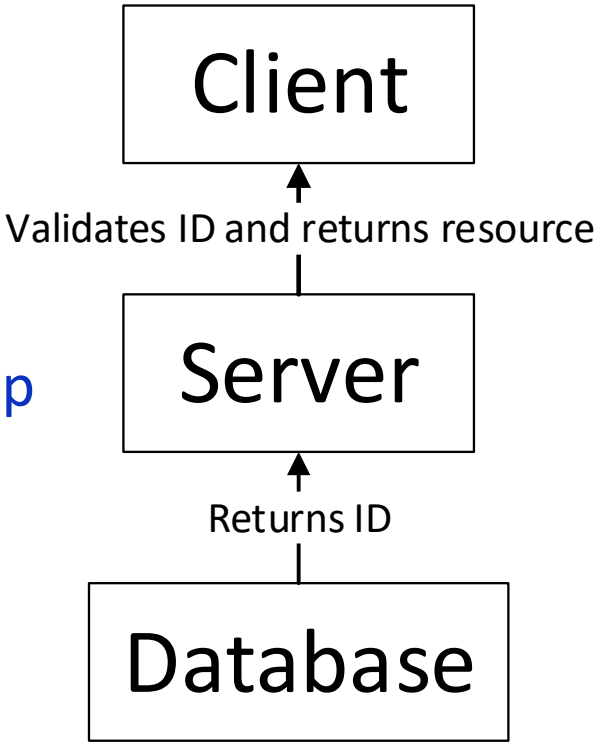
Stateless Servers

- An example:
- Server now forgets about the client
- Client makes a new request and uses ID
- Server has forgotten the client so looks it up



Stateless Servers

- An example:
- Server now forgets about the client
- Client makes a new request and uses ID
- Server has forgotten the client so looks it up
- Server finds ID and validates client
- Server sends back the resource
- Server forgets about client again



Stateful Servers

- Remember the client from request to request
- Keep, in memory, data about who the client is and what it is doing

- Connection may be kept open between client and server

- This means the server can respond quickly and doesn't need to rely on a database for users
- It also means that a multi-part 'conversation' can be maintained by the client and server

- Common for gaming

- Difficult to know when to delete the state.

UNIVERSITY of HULL

Stateless Servers vs. Stateful Servers

- So are stateless servers a waste of resources (e.g. server time/processing/database)?

- Yes: if server is only ever expected to work with a very small number of known clients
  - Server should remember and keep the state of its dealings with these clients
  - Faster / more efficient / customised to each client
  - You can create a stateful service/server (although ROA is probably not efficient in this situation)

- No: if server is expected to work with an arbitrary number of unknown clients
  - This is the majority of the web
  - Server resources are actually freed (managing 1000s+ of concurrent sessions would be very intensive, knowing when to end a session would be difficult, lots of resources used keeping state e.g. RAM)
  - Slower but more efficient for a larger number of clients

The In-Between

- Some servers store (cache) state for a finite amount of time where:
  - A follow-up request is likely to come quickly after a response
  - Speed is of the essence when it comes to responding to requests
  - Client numbers aren't likely to be too high, meaning that memory usage won't be large
  - The amount of data to be stored per client is small
- Sometimes, especially using HTTP, servers need to know who the client is to reconnect them to the stored state
  - This is often managed by session cookies, sent to the client and added to each subsequent request
    - Although these are sometimes used in a stateless scenario too for verification!
- Server waits for either a predefined time or a timeout (no further requests) and deletes the cached state.

Resource Oriented Architectures

- We live in a world of representation
  - We don't have enough time or resources to do everything ourselves, so we need to delegate some of our workload
  - We can't check all of the work someone else has done, so we rely on them being thorough and doing what we expect of them correctly
  - We may get a response after we request something from someone else – but this response is likely to be just the answer or a 'proof' rather than a full, detailed explanation of all the steps they took.

- Example: A bank transaction:
  - You want to buy something
  - You pay some money from your bank account to the merchant's bank account
  - You trust the banks to cooperate and manage the transfer on your behalf
  - You are given a verification 'proof' to let you know that the transfer was successful
  - You have no idea what ACTUALLY happened in the banks (it could have been someone walking some money between buildings or it could be 100% digital), but the proof is good enough representation for you to believe that the transfer worked/will work

UNIVERSITY of HULL

Resource Oriented Architectures

- The web works on representation (REQUEST: “do X” – RESPONSE: “yes/no/error”)
- The power of the web is in allowing other systems to do work that we can’t do or don’t have the resources for.
- We don’t expect to be given any more than a representation of what happened in any remote machine – be that some data that we requested or a verification that yes, some form data was submitted successfully.
- We don’t require information about exactly what is available at any time – software rarely needs to write itself so why rely on digital service contracts like WCF?
- We also don’t need to be certain what we’ll get back
  - we can work out what to do with what we get

UNIVERSITY of HULL

Resource Oriented Architectures

- Resource Oriented Architectures (ROAs) allow the use of resources, without any need for understanding or knowing what will happen in the remote machine.
- Usually a ROA is stateless, in that the server doesn't care what the client is doing and it doesn't remember the client directly
- This is (often) only possible by separation of the DATA from the Resource Oriented Server
  - One common way of doing this is by using a database
- If the client requires the server to 'remember' them, it's usually down to the client to store state information and send this to the server so that the server can pick back up from where it left off.
- Cookies/API Keys/Tokens/Etc…

UNIVERSITY of HULL

ROA vs. SOA

Key differences:

- SOA
  - Services that are independently deployable and loosely coupled, with well-defined interfaces.
  - Typically used in enterprise systems where a large number of services need to be integrated to provide complex business processes.
  - Focus is behaviours and functionality over data and information

- ROA
  - Architecture that primarily allows data management through a uniform interface (such as HTTP) and manipulation using standard methods (such as GET, PUT, POST, and DELETE).
    - This allows ‘everyone’ to speak the same language without the need of a controller (service bus).
  - Typically used in web-based applications and APIs where the focus is on accessing resources rather than executing complex business processes.

REST

- **RE**presentational **S**tate **T**ransfer
  - Architectural style, <u>not protocol</u>
  - Commonly ROA architectures are represented by REST architectures
- Stateless interaction
  - Every request must include all necessary information
  - Server forgets clients between calls
  - No ‘connection’
- Usually uses standard HTTP commands
  - GET, PUT, POST, DELETE, PATCH, Etc.
  - Meanings depend on application
- Different methods can be at different URIs

UNIVERSITY of HULL

REST

- REST was defined in 2000 by Roy Fielding
- Based on the original HTTP principles defined in 1994
- Importantly, the use of URIs (Uniform Resource Identifiers) allowed resources to be accessed over HTTP
- This structure reflects conventional folder/file access structures and is the basis of selecting information or making requests over the web

- A web application is a network of web resources that performs in a similar way to a state-machine. Requests are made by links (e.g. www.domain.com/api/myresource), and operations such as GET or PUT (state transitions)
- These operations may result in the resource being transferred to the requester or may cause some internal change

UNIVERSITY of HULL

Why REST?

- Cross-platform, standard protocols
  - REST is easily used by different applications on different devices
  - In a world with lots of devices with different OS's, programming languages and hardware from different manufacturers, that's very useful!!
- Simple, logical addressing scheme
  - http://supershop.com/orders/Recent/Feb/02/1234
  - http://supershop.com/orders/Recent/Feb/02/?customer=99
  - http://supershop.com/customers/9971
- Easy to implement as part of a web site (natively understood by browsers)
- Fits well with CRUD (Create, Read, Update, Delete) for data and database applications

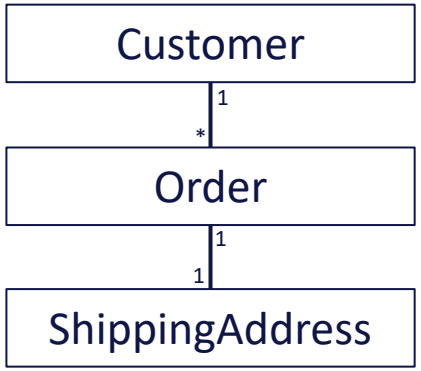
UNIVERSITY of HULL

Modelling the Data Domain

The REST architectural style fits best when modelling your data domain:

If your data schema is

- We have a collection of Customers
- Each Customer has a collection of Orders
- Each Order has a Shipping Address



The logical RESTful way to GET the Shipping Address for Customer 263's 5th Order is:

- Send a GET request to: www.mydomain.com/customer/263/order/5/ShippingAddress

Or to change the Shipping Address for this order:

- Send a POST request to: www.mydomain.com/customer/263/order/5/ShippingAddress

The data resource is identified in the URI
(Resource Oriented)

REST or SOAP?

REST	SOAP
Requires use of HTTP	Independent of protocol (e.g. HTTP)
Assumes transparent P2P connection (although this can be faked)	Works better through a service bus
Only an architectural pattern, not a standard	High level of standardisation
Fast and cheap	Can be slow and expensive
Minimal implementation cost, lean and simple	Bloated specification means difficult to implement and lots of ‘baggage’
Similar to the majority of other web technologies	Quite different to other web technologies
Compatible with anything that can use HTTP	Not always compatible
Can use a variety of message formats	Uses XML (verbose)

Web API

- ASP.NET-based
- HTTP only
  - Primarily JSON or XML media
  - Must be hosted (e.g. IIS)
- Request/response only
  - Can use WebSockets and SignalR for bidirectional
- No WSDL (but can use Odata or WADL for standardised creation and consumption)

- Better for REST than WCF?
  - WCF can be forced to use REST (although it is uncommon and complex)
  - This does give WCF the option of providing common services via SOAP ***or*** REST simultaneously, depending on the client's needs

UNIVERSITY of HULL

NO CONTRACT!

- No contract – What does this mean for our RESTful services?

- RESTful services **are not explicitly defined**
- We need some other way of defining them
- This is often managed differently by different service providers
- It is very common to have a website or a number of web pages dedicated to defining each element of a service, e.g. https://developers.google.com/maps/documentation/geocoding/start
  - These need to be understood by the developer and programmed in
  - RESTful services are not inherently self-descriptive
- There are some generic standards defined by various industries to create a common communication mechanism (but they are under no obligation to comply)
  - E.g. finance, travel, utilities, hotels, property

UNIVERSITY of HULL

Uniform Resource Identifiers

- A URI provides a compact, standard way of referring to a resource
- URI specifies
  - Resource type
  - Resource location
- Syntax:
  - <scheme name> : <hierarchical part> [? <query>] [# <fragment>]
  - Each part of hierarchy is split with /
  - Hierarchical part (path and resource) may end in file format type (e.g. .html, .jpg)
  - Query is sent to server for handling, Fragment is managed on client side
- Examples:
  - http://www.nowhere.com/index.html
  - http://www.google.co.uk/search?q=teryaki
  - http://en.wikipedia.org/wiki/Terry_Gilliam#Trivia
  - http://www.hive.hull.ac.uk/images/uni_vid.gif

UNIVERSITY of HULL

REST API

- Server interprets URI
  - Path data can include item identification
  - HTTP method invokes particular action depending on what parameters the method is expecting

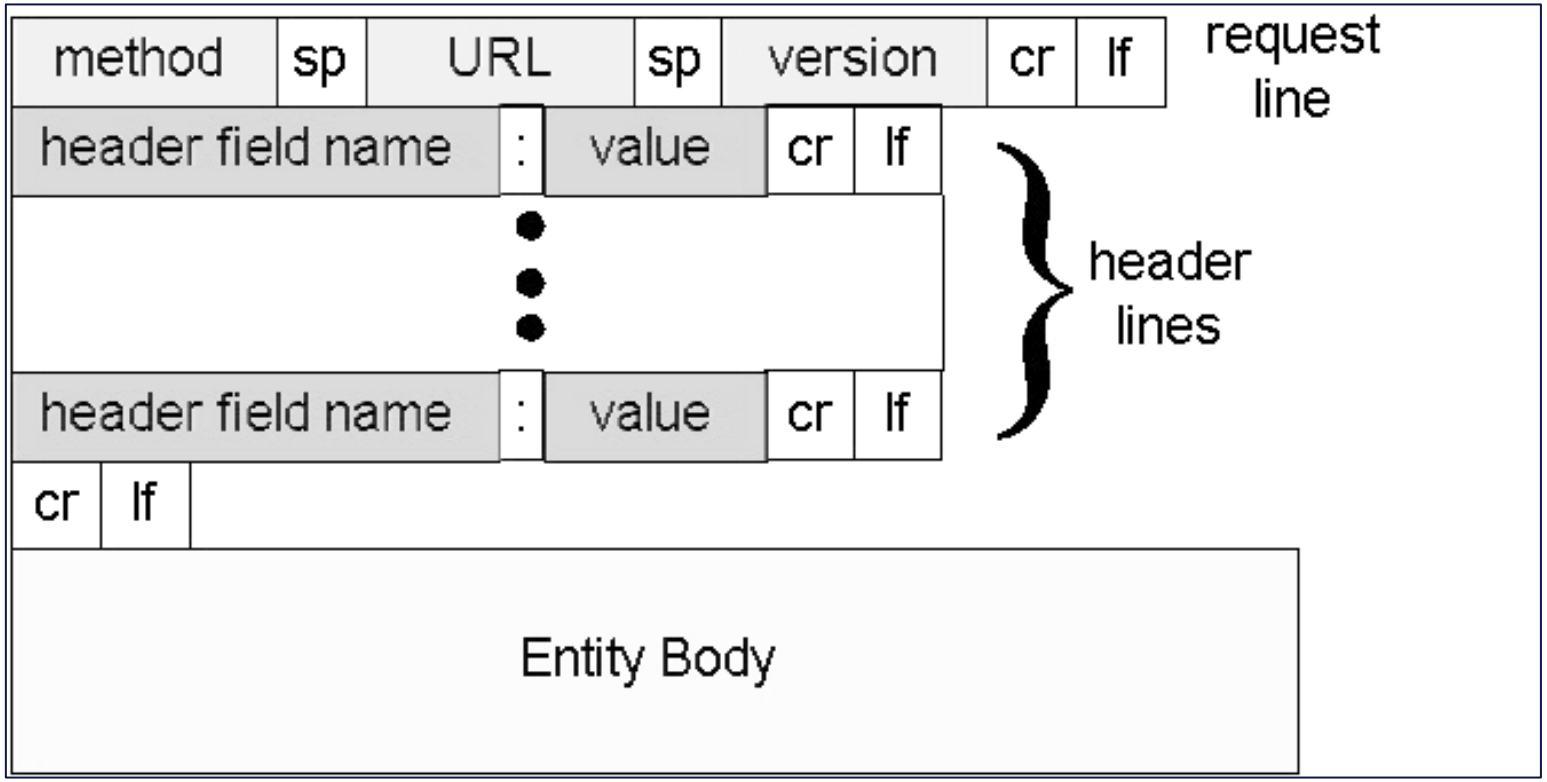
Target	GET	PUT	POST	DELETE
http://myDomain.net/puzzles	List all puzzles	Replace list of puzzles	Add a puzzle	Delete all puzzles
http://myDomain.net/puzzles/25	Return puzzle 25	Replace puzzle 25	Change puzzle 25	Delete puzzle 25

Note: in these examples for .NET WebAPI,
puzzles is the controller,
GET, PUT, POST and DELETE are HTTP Verbs
and 25 is the ID

UNIVERSITY of HULL

HTTP Messages

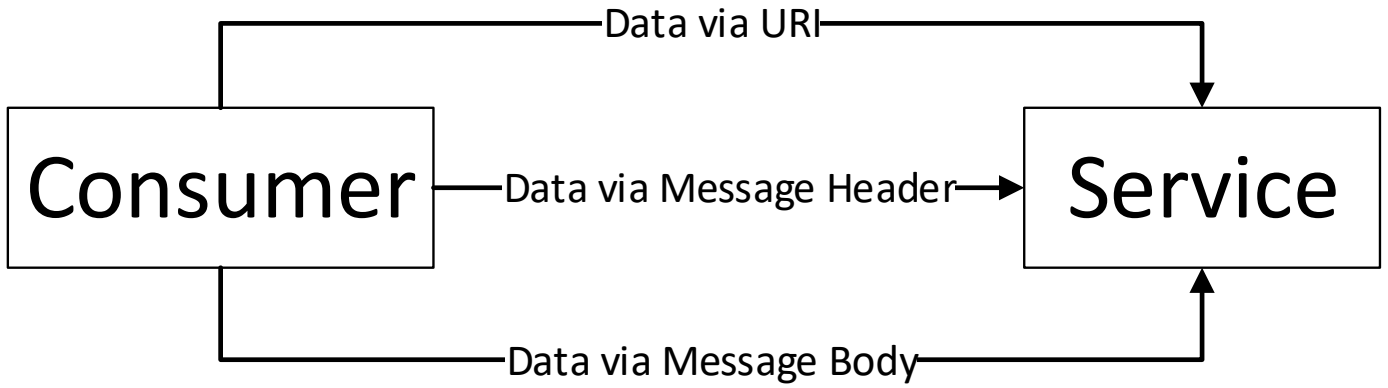
- URI tells the server what function is required
- As well as the URI, server has access to the HTTP message
  - Message gives additional detail to the server to perform the function

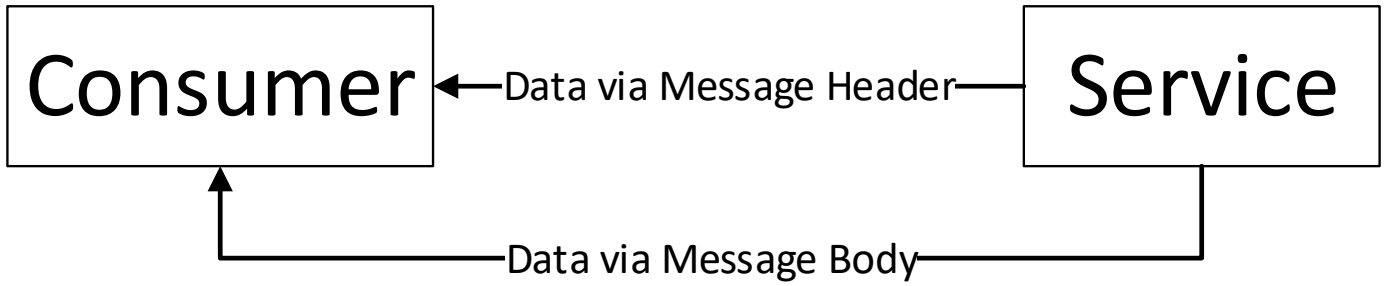


UNIVERSITY of HULL

RESTful Data Transfer

- Data can be provided in the Header or Body and extracted by the RESTful service for use
- Some HTTP requests require this to send data to a RESTful service (e.g. Post) some only rely on the URI (e.g. Get)
- The normal way to send data directly FROM a RESTful service is via an HTTP message





UNIVERSITY of HULL

RESTful Data Transfer Options -URI

- Consider this scenario:
  - You want to create a service which should allow validated consumer to get the address that is saved in the database for their user account

- Options for the REQUEST:

1. Require the consumer to craft a URI identifying what they want to do and supplying the data too:

   a) The service is expecting an ID number as a QUERY, so it extracts the user ID from the URI QUERY

      E.g. https://www.mydomain.com/api/account?ID=2378129

   b) Or the service is expecting an ID number as part of the PATH, so it extracts the user ID from the URI

      E.g. https://www.mydomain.com/api/account/2378129

Option B is arguably "*more RESTful*"

UNIVERSITY of HULL

RESTful Data Transfer Options - BODY

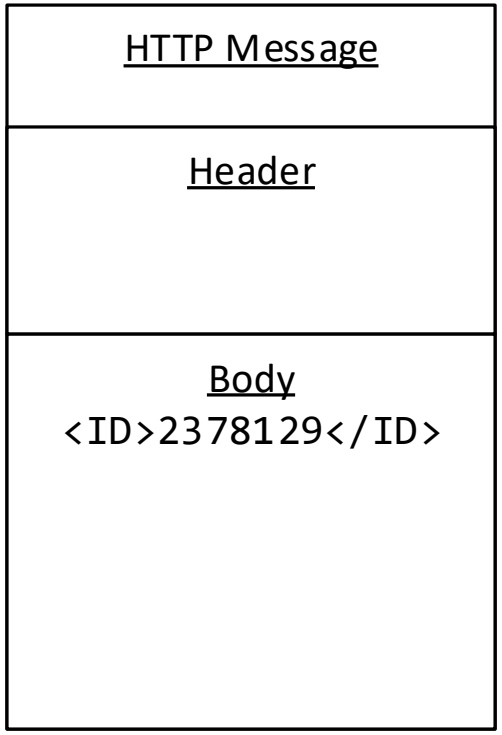
- Consider this scenario:
  - You want to create a service which should allow validated consumer to get the address that is saved in the database for their user account

- Options for the REQUEST:

2. Require the consumer to craft a URI identifying what they want to do and submit data in the request message body:

E.g. https://www.mydomain.com/api/account

The service is expecting an ID number in the message body
so it extracts the user ID from the body
(can use ANY format, although this format is basic XML)



RESTful Data Transfer Options - HEADER

- Consider this scenario:
  - You want to create a service which should allow validated consumer to get the address that is saved in the database for their user account

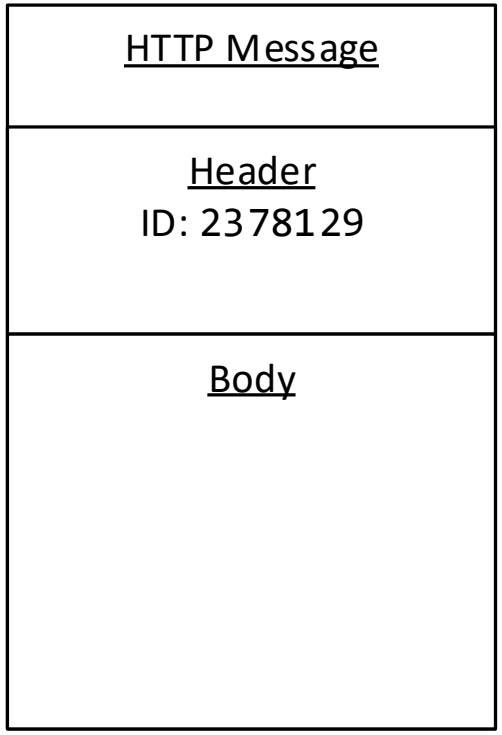
- Options for the REQUEST:

3. Require the consumer to craft a URI identifying what they want to do and submit data in the request message header:

E.g. https://www.mydomain.com/api/account

The service is expecting an ID number in the message header
so it extracts the user ID from the header
(standard header format is ‘<NAME>: <PARAMETER>’)

This is an unusual and considered bad practice as the header is not usually the correct place for data immediately relevant to the request



RESTful Data **Response** Options

- Consider this scenario:
  - You want to create a service which should allow validated consumer to get the address that is saved in the database for their user account

- Options for the RESPONSE:

  A URI isn't used for a response so the response **must** be managed by the HTTP response message

  It would be unconventional to return data like an address in a header
  (although it would be possible)

  Convention would be to use the message body.
  Again, this can be any data format (e.g. JSON, XML, plaintext, etc.)

UNIVERSITY of HULL

RESTful Reading

REST API Best Practices (free PDF):

- Https://github.com/tfredrich/RestApiTutorial.com/raw/master/media/RESTful%20Best%20Practices-v1_2.pdf

Building Web APIs with ASP.NET Core
Manning
Valerio De Sanctis

UNIVERSITY of HULL

Reflection

In this session we've looked at:

- Resource Oriented Architectures
- Stateful vs. stateless services/servers
- RESTful services
- Web API (briefly)
- Using REST via HTTP principles

In the next session we will delve more into detail on .NET Web API and creating/consuming RESTful services

UNIVERSITY of HULL